

Laborator 9- Algoritmi

1. Pentru un graf reprezentat prin liste de adiacență, implementați parcurgerile în lățime și adâncime, procedura care afișează calea cea mai scurtă de la un nod la altul, sortarea topologică și determinarea componentelor tare conexe. Lucrați cu fișiere header.

Modul de implementare a unei liste de adiacență (vezi Curs 9 – slide 5):

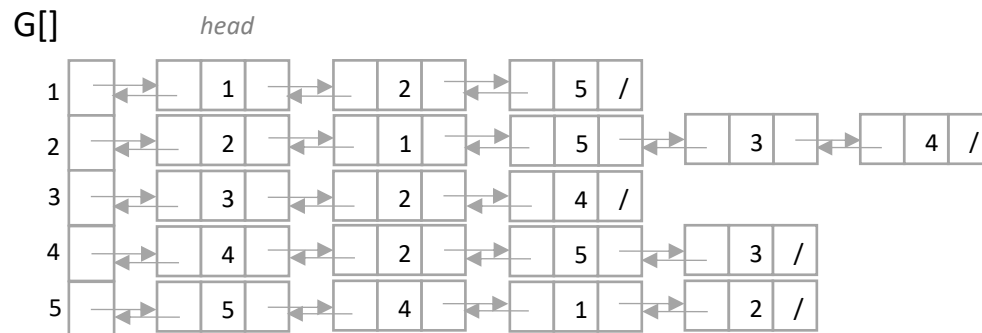
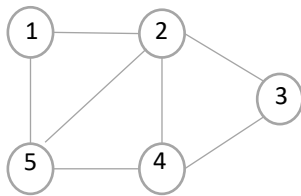
```
typedef struct nod_lista {
    enum {white, gray, black} color;
    int d, f;
    int cheie;
    struct nod_lista *next, *prev;
} t_nod_lista;

typedef struct {
    t_nod_lista *head;
} t_lista;
```

- Lista de adiacență este o listă dublu înlanțuită
- Operațiile de inserare/ștergere/creare sunt identice ca și la liste
- *prev este câmpul *pi din pseudocod
- În main, lista de adiacență este similară cu un hashtable implementat prin liste:

```
t_lista G[100];
```

Modul de parcurgere a nodurilor într-un graf printr-o listă de adiacență este:



```
NOD_LISTA nod;
//pentru fiecare nod in G[]
FOR i:=1 TO N
    // ne pozitionam in capul listei vecinilor
    nod := G[i].head
    //parcurge vecini(i) in lista dublu inlantuita
    WHILE nod <> NULL
        //..... // prelucrari pe nod
        //navigare la urmatorul nod
        nod := nod->next
    END WHILE
END FOR
```

Pseudocod:

<pre> BFS(LISTA_ADJACENTA G, S) Array Q FOR each vertex u in G IF u <> S THEN u.color := WHITE u.d := ∞ u.prev := NULL END IF END FOR S.color := GRAY S.d := 0 S.prev := NULL PRINT S.cheie MARK_VISITED(G, S.cheie) ENQUEUE(Q, S) //nodurile vizitate in coada WHILE Q <> ∅ DO u := DEQUEUE(Q) FOR each vertex v in G.Adj[u] IF v.color == WHITE THEN v.color := GRAY v.d := u.d + 1 v.prev := u PRINT v.cheie MARK_VISITED(G, v.cheie) ENQUEUE(Q, v) END IF END FOR u.color := BLACK END WHILE END BFS </pre>	<pre> INIT_LISTA_AD(LISTA_ADJACENTA G) FOR i:=1 TO N MAKENULL(G[i]) //vezi liste END FOR END INIT_LISTA_AD INSERT_VECINI(LISTA_ADJACENTA G, S) NOD_LISTA nod PRINT "Nod sursa "+S+": " G[S].head.cheie := S PRINT "Dati vecinii nodului sursa: " DO PRINT "Vecin cu "+S READ v IF v > 0 AND v <= N AND v <> S THEN Allocate mem. nod nod.cheie := v LIST_INSERT(G[S], nod) END IF WHILE v > 0 END INSERT_VECINI PRINT_LISTA(LISTA_ADJACENTA G) NOD_LISTA nod FOR i:=1 TO N nod := G[i].head.next PRINT i + ": " WHILE nod <> NULL DO PRINT nod.cheie nod := nod.next END WHILE PRINT NEW LINE END FOR END PRINT_LISTA </pre>
<pre> MARK_VISITED(LISTA_ADJACENTA G, KEY) FOR each vertex u in G IF u.cheie == KEY u.color := GRAY END IF END FOR END MARK_VISITED </pre>	<pre> PRINT_PATH(LISTA_ADJACENTA G, S, D) IF V==S THEN PRINT S.key ELSE IF V.prev == NULL THEN PRINT "nu exista cale" ELSE PRINT_PATH(G, S, V.prev) PRINT V.key END IF END PRINT_PATH </pre>

<pre> DFS_VISIT(LISTA_ADIACENTA G, U) time := time + 1 //vertex u -discovered U.d := time U.color := GRAY PRINT U.key MARK_VISITED(G,U.cheie) FOR each vertex v in G.Adj[U] //edge (u,v) IF v.color == WHITE THEN v.prev := U DFS_VISIT(G, v) END IF END FOR U.color := BLACK time := time + 1 U.f := time END DFS_VISIT </pre>	<pre> DFS(LISTA_ADIACENTA G) FOR each vertex u in G u.color := WHITE u.prev:= NULL END FOR time := 0 FOR each vertex u in G IF u.color == WHITE THEN DFS_VISIT(G, u) END IF END FOR END DFS </pre>
<pre> DFS_VISIT_TOP(G, U, topo) time := time + 1 U.d := time U.color := GRAY MARK_VISITED(G, U.cheie) FOR each vertex v in G.Adj[U] IF v.color == WHITE THEN v.prev := U DFS_VISIT_TOP(G, v, topo) END IF END FOR U.color := BLACK time := time + 1 U.f := time LIST_INSERT(topo, U) //vezi liste END DFS_VISIT_TOP </pre>	<pre> MAIN() LISTA_ADIACENTA G LISTA topology PRINT "Nr noduri=" READ N INIT_LISTA_AD(G) FOR i:=1 TO N INSERT_VECINI(G, i) END FOR PRINT "Lista adiacenta: " PRINT_LISTA(G) PRINT "Parcuregere in latime: " PRINT "Nod start: " READ S BFS(G, S) PRINT "Calea cea mai scurta: " PRINT "Nod start: " READ n1 PRINT "Nod final: " READ n2 PRINT_PATH(G, n1, n2) PRINT "Parcuregere in adancime: " DFS(G) PRINT "Sortare topologica: " topology:=TOP_SORT(G) LIST_PRINT(topology) //vezi liste //SC_COMPONENTS(G) END MAIN </pre>
<pre> TOP_SORT(LISTA_ADIACENTA G) LISTA topology MAKENULL(topology) FOR each vertex u in G u.color := WHITE u.prev := NULL END FOR time := 0 FOR each vertex u in G IF u.color == WHITE THEN DFS_VISIT_TOP(G, u, topology) END IF END FOR RETURN topology END TOP_SORT </pre>	

Tema: Implementați procedura SC_COMPONENTS de determinare a componentelor tare-conexe dintr-un graf orientat:

1. se apelează DFS(G) pentru a calcula U.f pentru fiecare nod timpurile finale
2. calculează $G^T = G$ transpus – $G^T = (V, E^T)$, unde E^T reprezintă muchiile grafului G cu direcțiile inversate; ex: dacă există arc de la 1 la 2 atunci în G^T acesta devine arc de la 2 la 1. (vezi Cormen de la pag. 615)
3. apelăm o procedură DFS_GT(G^T) pentru graful transpus G^T dar procedura DFS_GT o implementăm similar ca și DFS cu excepția faptului ca în bucla principală (al doilea FOR din DFS) nodurile se consideră în ordine descrescătoare a U.f (se poate face sortarea prealabilă a nodurilor în funcție de U.f) => pădure de arbori în care fiecare arbore reprezintă câte o componentă tare-conexă
4. parcurgem pădurea de arbori formată la punctul 3 (prin depth-first) și afișăm fiecare nod al fiecărui arbore- fiecare arbore reprezentând câte o componentă tare-conexă.

Probleme optionale:

1. Pentru un graf reprezentat prin liste de adiacență, scrieți o procedură nerecursivă pentru parcurgerea în adâncime
2. O altă modalitate de a determina sortarea topologică a unui graf este să găsim, în mod repetat, nodul care are in-degree 0 (adică nici un arc nu intră în nod), apoi printăm acest nod și după aceea îl ștergem din graf, împreună cu toate nodurile care ies din acest nod. Implementați acest algoritm. Explicați ce se întâmplă dacă graful are cicluri.